

2.

(b)

```
abc 01> read lsv/pa1/mul.blif
abc 02> print_stats
mul                                     : i/o =   4/   4 lat =   0 nd =   8 edge =   16 cube =   10 lev = 3
abc 02> show
abc 02> strash
abc 03> show
abc 03> collapse
abc 04> show_bdd -g
```

Figure 1. ABC output after reading `mul.blif` and executing `print_stats`.

After reading the BLIF file, `print_stats` produced:

i/o = 4/4 lat = 0 nd = 8 edge = 16 cube = 10 lev = 3

The circuit contains four primary inputs, four primary outputs, eight logic nodes, sixteen edges, ten SOP cubes, and three logic levels. It has no latches because the multiplier is a combinational circuit.

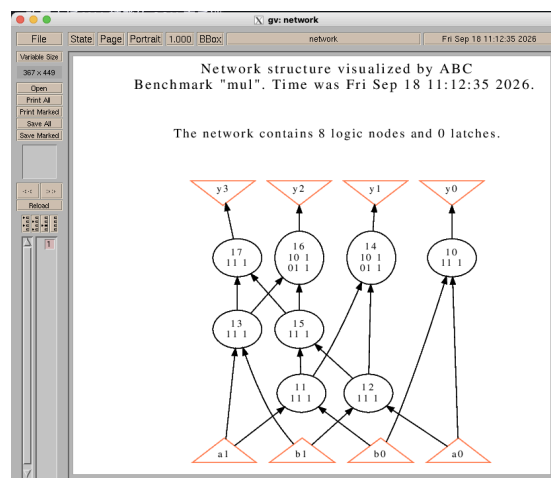


Figure 2. Original logic network of the two-bit unsigned multiplier.

The four triangles at the bottom represent inputs `a1`, `b1`, `b0`, and `a0`. The four triangles at the top represent outputs `y3`, `y2`, `y1`, and `y0`.

The network contains eight logic nodes. The values inside each node describe its local SOP function. For example, `11 1` represents an AND function, while `10 1` and `01 1` together represent an XOR function.

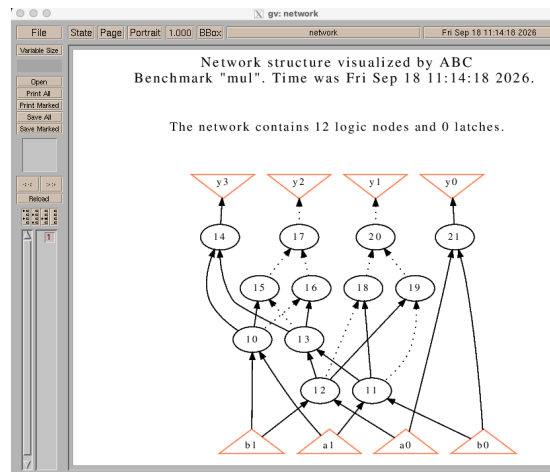


Figure 3. Structurally hashed AIG of the two-bit unsigned multiplier.

The resulting AIG contains twelve logic nodes. In this representation, the circuit is constructed using two-input AND nodes and complemented edges. The number of nodes increases because an XOR function requires multiple AIG nodes.

Structural hashing also allows identical AIG structures to share the same node.

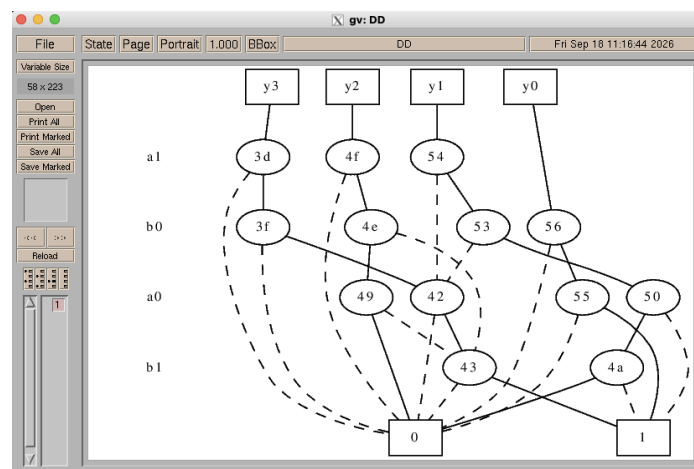


Figure 4. BDDs of all four multiplier outputs generated by `show_bdd -g`.

The four boxes at the top represent outputs **y3**, **y2**, **y1**, and **y0**. The oval nodes are BDD decision nodes, while the boxes labeled **0** and **1** are terminal nodes.

The labels on the left show the BDD variable order. Solid and dashed lines represent the two possible branches of each decision. Some BDD nodes are shared because they represent equivalent subfunctions.

3.

(a)

1. aig vs. strash

```
abc 01> read lsv/pa1/mul.blif
abc 02> aig
abc 02> print_stats
mul                      : i/o =   4/   4  lat =   0  nd =    8  edge =   16  aig =   12  lev = 3
abc 02> show
abc 02> read lsv/pa1/mul.blif
abc 03> strash
abc 04> print_stats
mul                      : i/o =   4/   4  lat =   0  and =   12  lev = 4
abc 04> show
```

Figure 5. ABC commands and statistics for the AIG and structurally hashed AIG.

After executing **aig**, ABC reported:

i/o = 4/4 lat = 0 nd = 8 edge = 16 aig = 12 lev = 3

The network still contains eight logic nodes. Their local functions are represented by a total of twelve AIG nodes.

After executing **strash**, ABC reported:

i/o = 4/4 lat = 0 and = 12 lev = 4

The complete network is converted into a structurally hashed AIG containing twelve AND nodes and four logic levels.

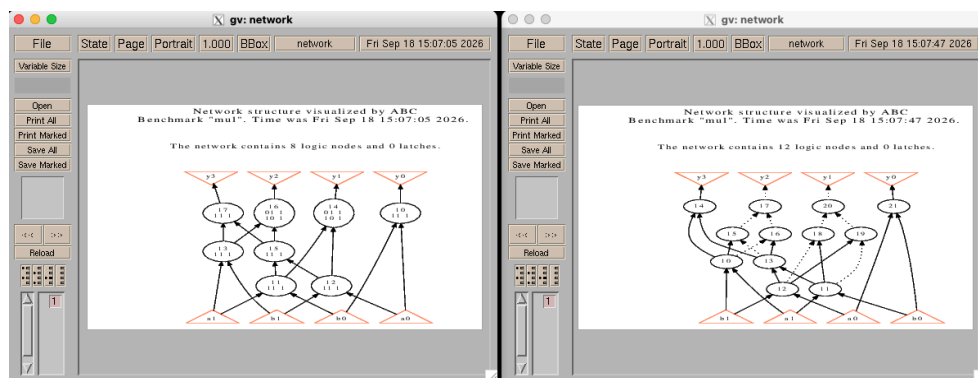


Figure 6. AIG logic network (left) and structurally hashed AIG (right).

The left side of Figure 6 is the result of **aig**. It preserves the original network with eight logic nodes, while the function of each node is represented internally as an AIG.

The right side is the result of **strash**. It converts the complete circuit into two-input AND nodes and complemented edges. Structurally identical operations can share the same AIG node.

Therefore, **aig** changes the representation of the local node functions, while **strash** converts the entire network into one structurally hashed AIG.

2. bdd vs. collapse

```

abc 01> read lsv/pa1/mul.blif
abc 02> bdd
abc 02> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 bdd = 16 lev = 3
abc 02> show
abc 02> read lsv/pa1/mul.blif
abc 03> collapse
abc 04> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 4 edge = 14 bdd = 14 lev = 1
abc 04> show_bdd -g

```

Figure 7. Commands and statistics for the BDD logic network and collapsed BDD.

After executing **bdd**, ABC reported:

i/o = 4/4 lat = 0 nd = 8 edge = 16 bdd = 16 lev = 3

The network contains eight logic nodes, sixteen edges, sixteen BDD nodes, and three logic levels. The **bdd** command represents the local function of each logic node using a BDD, while preserving the original multi-level network structure.

After executing **collapse**, ABC reported:

i/o = 4/4 lat = 0 nd = 4 edge = 14 bdd = 14 lev = 1

The collapsed network contains only four logic nodes, corresponding to the four primary outputs. The number of logic levels decreases from three to one because the intermediate nodes are removed. Each output is represented directly as a function of the primary inputs.

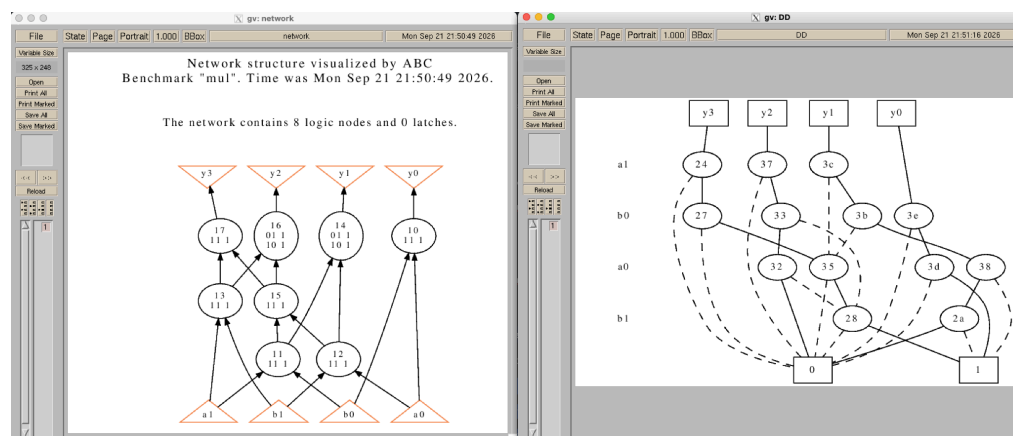


Figure 8. BDD logic network (left) and collapsed BDD representation (right).

The left side of Figure 8 shows the result of the **bdd** command. The original eight-node, three-level network is preserved, but the local function of each node is represented using a BDD.

The right side shows the result of **collapse** and **show_bdd -g**. The four boxes at the top represent outputs **y3**, **y2**, **y1**, and **y0**. The oval nodes are BDD decision nodes, while the

boxes labeled 0 and 1 are terminal nodes. The labels on the left indicate the variable order used by the BDD.

The main difference is that **bdd** constructs BDDs for local node functions, while **collapse** constructs the global function of each primary output. Therefore, **bdd** preserves the original multi-level network, whereas **collapse** removes the intermediate logic and reduces the network to one logic level.

(b)

```
abc 01> read lsv/pa1/mul.blif
abc 02> strash
abc 03> logic
abc 04> sop
abc 04> print_stats
mul                                     : i/o =   4/   4 lat =   0 nd =   12 edge =   24 cube =   12 lev =  4
abc 04> show
```

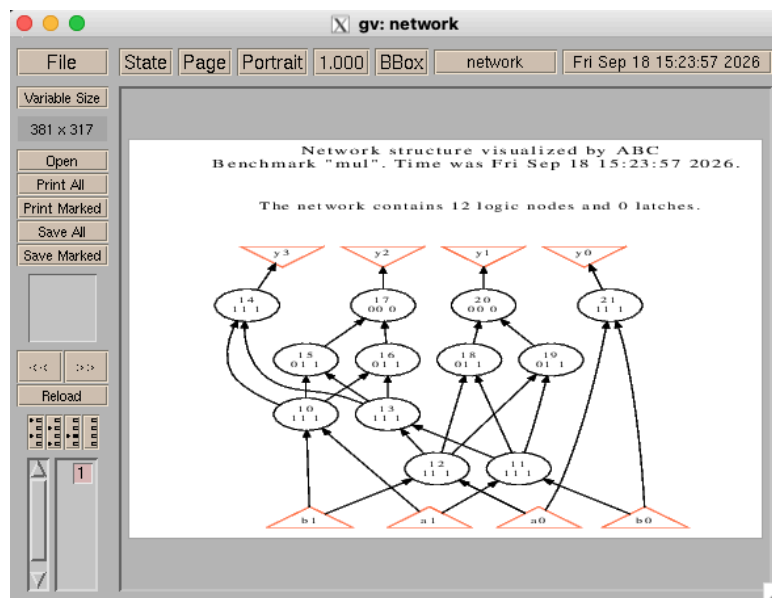


Figure 9. Logic network with node functions represented in SOP form.

The expressions displayed inside the nodes confirm the SOP representation. For example,

11 1

means that the output is one when both inputs are one. Other cubes, such as 00 1 and 01 1, represent product terms with complemented input variables.

Thus, the command sequence

strash → logic → sop

successfully converts a structurally hashed AIG into a logic network whose node functions are represented in SOP form.